Dataset Notes: C-MAPSS
Turbofan Engine Degradation Simulation Data

Saxena, Goebel, Simon, Eklund — PHM'08 dataset — *Working reference edition*

Contents

Overview

The C-MAPSS dataset is the data product of the paper *Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation* (Saxena et al., PHM'08). It is one of the most widely used benchmarks in predictive maintenance and Remaining Useful Life (RUL) research.

This document is a working reference for the dataset files themselves — their structure, content, and how to use them. It is intentionally separate from the paper notes. Understanding the simulation methodology is one thing; working with the actual files is another.

> **What the dataset is:** A collection of multivariate time series, each representing one simulated aircraft engine's sensor readings from initial operation through failure. The task is to predict how many cycles remain before failure, given only the sensor readings up to the current cycle.

The Four Sub-Datasets

The C-MAPSS release is not one dataset but four, named FD001 through FD004. They share the same 26-column file format and the same engine model, but differ along two independent axes of difficulty.

Figure 1 shows the complete overview.

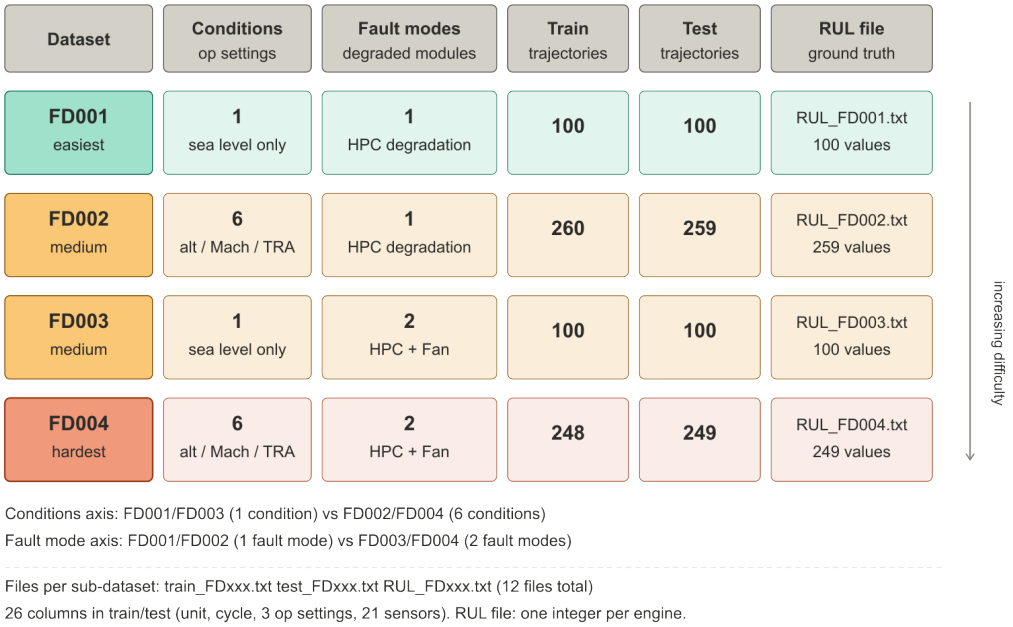

Figure 1: The four C-MAPSS sub-datasets. Color encodes difficulty: teal = easiest, amber = medium, coral = hardest. The difficulty arrow on the right shows the overall ordering.

Two independent axes of difficulty

Conditions axis separates FD001/FD003 (1 operating condition: sea level) from FD002/FD004 (6 operating conditions: combinations of altitude, Mach number, and TRA). With multiple conditions, raw sensor values are not comparable across cycles because the same sensor reads differently at different flight points. This requires normalization before any analysis.

Fault mode axis separates FD001/FD002 (HPC degradation only) from FD003/FD004 (HPC degradation plus Fan degradation). With two fault modes, the algorithm does not know which component is degrading. No label is provided.

	FD001	FD002	FD003	FD004
Conditions	1	6	1	6
Fault modes	1	1	2	2
Train	100	260	100	248
Test	100	259	100	249
Difficulty	lowest	medium	medium	highest

FD002 vs FD003 are incomparable. Both are "medium" difficulty but hard in different ways. FD002 is hard because of operating condition variability. FD003 is hard because of fault mode ambiguity. An algorithm that handles FD002 well may still fail on FD003, and vice versa.

Practical warning: Most published results report on FD001 only — it is the easiest, the most reproducible, and the most cited. A result on FD001 alone should be interpreted cautiously. It says nothing about robustness to condition variability or fault mode ambiguity.

File Structure: 12 Files

Each sub-dataset produces exactly three files, giving $4 \times 3 = \mathbf{12}$ files in total:

File	Contents
`train_FDxxx.txt`	Full run-to-failure trajectories. Each engine runs from its initial condition to failure. The last row of each engine is the failure cycle. **RUL is not provided** — it must be computed from the data: $\text{RUL}(t) = t_{\max} - t$.
`test_FDxxx.txt`	Truncated trajectories. Each engine's time series stops at some point before failure. The last row is *not* the failure cycle. **RUL is unknown** and must be predicted.
`RUL_FDxxx.txt`	Ground truth for the test set. One integer per engine: the true RUL at the last observed cycle of that engine. Used only for evaluation.

Figure 2 shows how the three files relate to a single conceptual engine life.

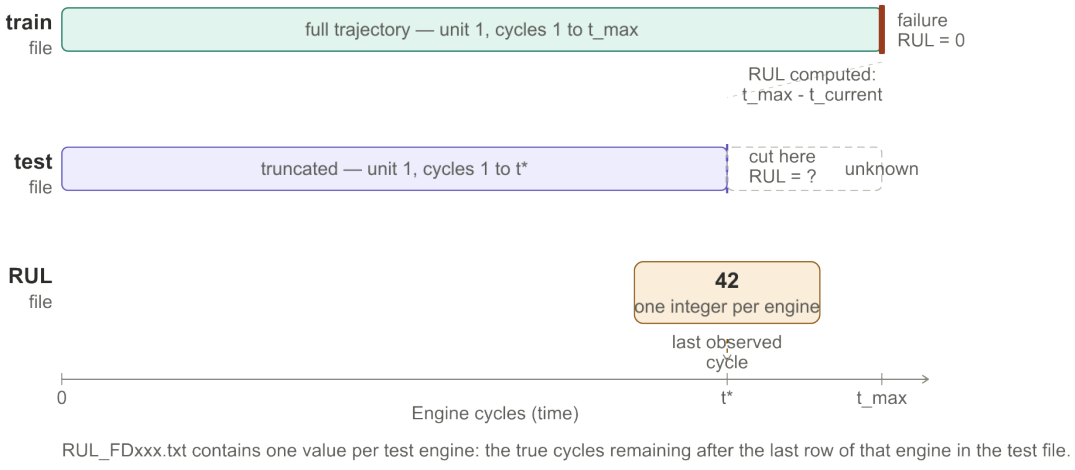

Figure 2: The relationship between the three file types for one engine. The train file covers the full life to failure. The test file stops at t^*. The RUL file provides one number — the true cycles remaining at t^*.

All `train` and `test` files are plain text, space-separated, with no header row. The `RUL` files are a single column of integers, one per engine, also with no header.

The 26-Column Format

Every row in a `train` or `test` file is one cycle of one engine. The 26 columns are fixed across all four sub-datasets.

Table 1: Complete column reference for C-MAPSS train and test files. This table is your Excel header row.

Col	Name	Units	Description
1	`unit_number`	—	Engine ID. Integer, resets per file.
2	`time_cycles`	cycles	Cycle counter per engine. Starts at 1.
3	`op_setting_1`	ft	Altitude.
4	`op_setting_2`	—	Mach number.
5	`op_setting_3`	—	Throttle resolver angle (TRA).
6	`T2`	°R	Total temperature, fan inlet.
7	`T24`	°R	Total temperature, LPC outlet.
8	`T30`	°R	Total temperature, HPC outlet.
9	`T50`	°R	Total temperature, LPT outlet.
10	`P2`	psia	Pressure, fan inlet.
11	`P15`	psia	Total pressure, bypass duct.
12	`P30`	psia	Total pressure, HPC outlet.
13	`Nf`	rpm	Physical fan speed.
14	`Nc`	rpm	Physical core speed.
15	`epr`	—	Engine pressure ratio (P_{50}/P_2).
16	`Ps30`	psia	Static pressure, HPC outlet.
17	`phi`	pps/psi	Fuel flow / P_{s30} ratio.
18	`NRf`	rpm	Corrected fan speed.
19	`NRc`	rpm	Corrected core speed.
20	`BPR`	—	Bypass ratio.
21	`farB`	—	Burner fuel-air ratio.
22	`htBleed`	—	Bleed enthalpy.
23	`Nf_dmd`	rpm	Demanded fan speed.
24	`PCNfR_dmd`	rpm	Demanded corrected fan speed.
25	`W31`	lbm/s	HPT coolant bleed.
26	`W32`	lbm/s	LPT coolant bleed.

The column grouping is meaningful: columns 1–2 are identifiers, columns 3–5 are operating conditions, and columns 6–26 are the 21 sensor measurements.

Computing the Training RUL

The training files do not contain a RUL column. It must be derived from the data. The standard procedure is:

1. Group all rows by `unit_number`.
2. For each engine, find $t_{\max}$ — the largest value of `time_cycles` for that engine. This is the failure cycle.
3. For every row of that engine: RUL $= t_{\max} -$ `time_cycles`

This gives RUL $= 0$ at the failure cycle, counting upward toward earlier cycles. Every engine in the training set ends with RUL $= 0$.

The piece-wise linear RUL cap

A common preprocessing convention — not specified in the dataset or paper — is to **cap the RUL** at a maximum value (typically 125 cycles) for early cycles. The target shape becomes piece-wise linear: flat at the cap value, then linearly decreasing to zero.

Figure 3 shows the two shapes side by side.

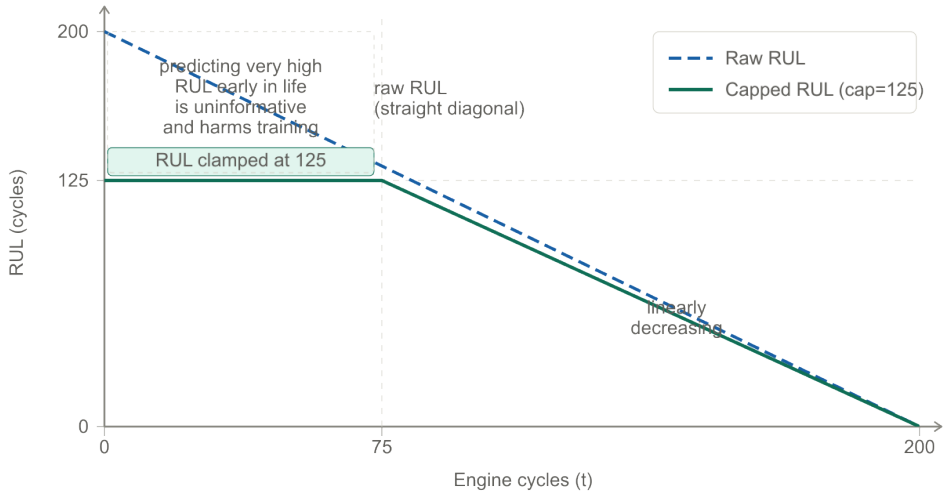

Figure 3: Raw RUL (dashed blue) vs piece-wise linear capped RUL (solid teal, cap = 125). The cap removes the flat region at the top where precise long-range prediction is both impossible and uninformative.

Why cap the RUL? Early in an engine's life, the sensor signals carry little degradation information — the engine is healthy and its behaviour is dominated by operating condition variability, not wear. Asking a model to predict RUL = 180 precisely from a healthy-looking signal is unreasonable and wastes model capacity. The cap replaces this impossible task with a flat "healthy" label, letting the model focus its capacity on the degradation phase where prediction actually matters.

The cap value of 125 is a convention, not a physical constant. Different papers use values from 100 to 150. When comparing results across papers, always check whether a cap was applied and what value was used.

The Multi-Condition Problem

For FD002 and FD004, the six operating conditions cause a problem that is not obvious until you look at the raw data.

The problem

Each of the six flight conditions produces a distinct range of absolute sensor values. Fan inlet temperature T_2 at sea level is hundreds of degrees Rankine higher than at 40,000 ft. When an engine cycles through different flight conditions from one cycle to the next, the raw sensor readings jump between these bands — not because of degradation, but simply because of where the engine is operating.

Figure 4 shows this effect clearly.

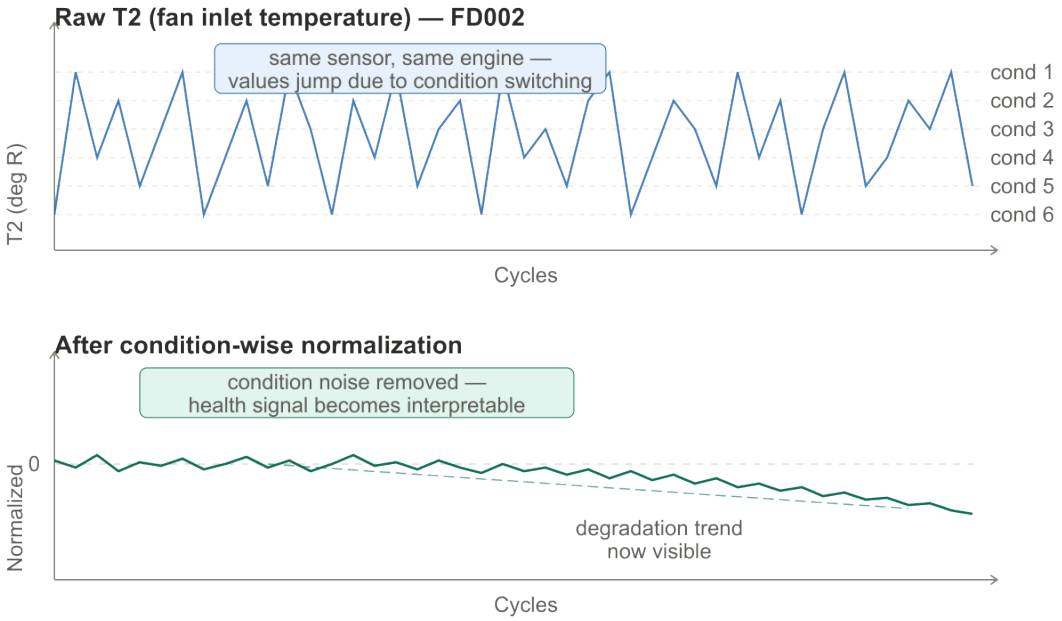

Figure 4: Top: raw T_2 values for one engine in FD002. The signal jumps between six bands due to condition switching — not degradation. Bottom: the same signal after condition-wise normalisation. The condition bands collapse and a slow degradation drift becomes visible.

Feeding raw sensor values from FD002 or FD004 into an ML model trains the model to recognise operating conditions, not degradation. The condition signal completely overwhelms the health signal.

The solution: condition-wise normalisation

The standard fix is to normalize each sensor reading relative to its expected healthy value *at that specific operating condition.* Figure 5 shows the full pipeline.

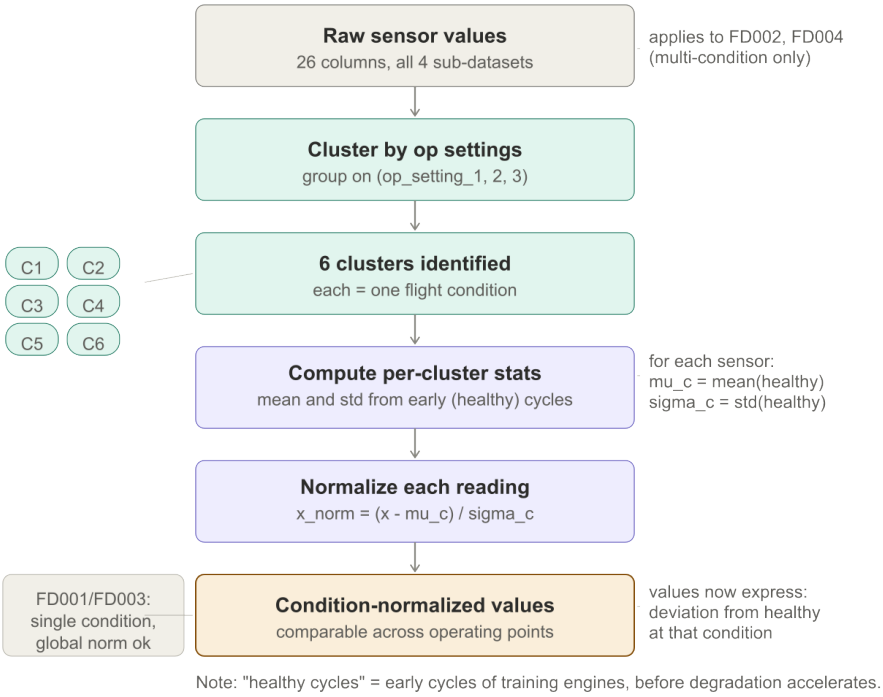

Figure 5: The condition-wise normalisation pipeline for FD002 and FD004. Six clusters are identified from the three operating setting columns. Each sensor is normalised within its cluster using statistics computed from the healthy (early) cycles.

The procedure step by step:

1. **Cluster** all cycles by operating condition using the three `op_setting` columns. Six distinct clusters emerge naturally for FD002 and FD004.
2. **Compute baseline statistics** for each sensor within each cluster, using only the early (healthy) cycles of training engines:

$$\mu_{s,c} = \text{mean of sensor } s \text{ in cluster } c \text{ over healthy cycles}$$

$$\sigma_{s,c} = \text{std of sensor } s \text{ in cluster } c \text{ over healthy cycles}$$

3. **Normalise** every reading:

$$x_{\text{norm}} = \frac{x - \mu_{s,c}}{\sigma_{s,c}}$$

After this step, each sensor value expresses deviation from the healthy baseline at that operating condition. Values near zero mean healthy behaviour; drift away from zero indicates degradation.

For FD001 and FD003 (single condition), this step simplifies to a global normalisation — one mean and standard deviation per sensor across all training cycles. The formula is the same; only one cluster is needed.

What counts as "healthy cycles"? There is no official definition. Common choices are: the first N cycles of each training engine (e.g. the first 30), or all cycles of training engines with RUL $\geq$ some threshold (e.g. RUL ≥ 125). The choice affects the baseline statistics and

therefore the normalized values. When comparing preprocessing approaches across papers, this is a key parameter to check.

What the Data Does Not Contain

It is worth being explicit about what is absent from the files, because the simulation paper describes quantities that never appear in the data:

Absent	Implication
Health index $H(t)$	Never provided. Must be inferred from sensors.
Margin values (SmFan, SmHPC, ...)	Used only inside the simulation. Not in files.
Hidden state $(e(t), f(t))$	The actual degradation trajectory is never revealed.
Fault mode label	FD003/FD004 do not indicate which module is degrading.
Operating condition label	The condition identity must be inferred from `op_setting_1/2/3`.
RUL column in training files	Must be computed: $t_{\max} - t$.

The algorithm sees only the 26 raw columns. Every other quantity is part of the inference problem.

Quick Reference

File naming

File	Purpose
`train_FD001.txt`	Training data, easiest sub-dataset
`test_FD001.txt`	Test data, easiest sub-dataset
`RUL_FD001.txt`	Test ground truth, easiest sub-dataset
`train_FD002.txt`	Training data, 6 conditions
`test_FD002.txt`	Test data, 6 conditions
`RUL_FD002.txt`	Test ground truth, 6 conditions
`train_FD003.txt`	Training data, 2 fault modes
`test_FD003.txt`	Test data, 2 fault modes
`RUL_FD003.txt`	Test ground truth, 2 fault modes
`train_FD004.txt`	Training data, hardest sub-dataset
`test_FD004.txt`	Test data, hardest sub-dataset
`RUL_FD004.txt`	Test ground truth, hardest sub-dataset

Key numbers to remember

Quantity	Value
Total files	12
Columns per row	26
Identifier columns	2 (unit, cycle)
Operating settings	3 (cols 3–5)
Sensor columns	21 (cols 6–26)
Operating conditions	1 (FD001/FD003) or 6 (FD002/FD004)
Fault modes	1 (FD001/FD002) or 2 (FD003/FD004)
Common RUL cap	125 cycles (convention, not specified)